

module-15-time-scheduling

Chapter 15: Keeping Time, Scheduling Tasks, and Launching Programs | 37 programs in learning order

Source: Automate the Boring Stuff with Python 1e, Al Sweigart (CC-BY-NC-SA). Syllabus Ch15: time, stopwatch, datetime, threading, subprocess launch, countdown

How to use this guide (beginner path)

1) Read Concepts once. 2) Study the Visuals - redraw each on paper. 3) Follow the Program map. 4) Work programs 01..N in order. [shell]: type every plain line into IDLE, compare with # OUT: comments (errors shown are intentional demos). [script]: save, predict output, run with F5. 5) Drill Interview prep. 6) Teach it: explain each visual aloud without looking.

Concepts in detail

`time.time()` stamps seconds (subtract for durations); `time.sleep(n)` pauses; `round(x, 2)` tidies display - the stopwatch project laps on Enter. `datetime` builds/compares calendar moments; `timedelta` does date math; `strftime()` formats, `strptime()` parses. `threading` runs functions simultaneously (start, pass args, join to wait). `subprocess.Popen` launches other apps/files/scripts; the countdown + alarm chains sleep with sound.

Key ideas to carry forward

- `start = time.time(); ...; time.time() - start` -> elapsed seconds.
- `datetime(2015,10,31,12,0,0) + timedelta(days=30)` -> date math.
- `Thread(target=func, args=[...]).start(); threads[i].join()` to wait for all.

Classic beginner mistakes

- ! Sleep-based timing drifting - measure with `time()`, don't count sleeps.
- ! Threads sharing one mutable list without locks (race conditions).

Study order hint: 01-08 time + stopwatch, 09-17 datetime, 18-26 threading + threaded downloader, 27-37 launching + countdown.

Interview prep - questions you must answer cold

Say each answer aloud in under 60 seconds, then prove it with one program from this module.

Q1. Measure elapsed time?

A: `start = time.time(); ...; time.time() - start`. Never count sleeps - measure the clock.

Q2. `sleep()` uses?

A: Pauses seconds (floats ok) for pacing, polling, countdowns, rate-limiting.

Q3. `datetime` vs `time`?

A: `time` = epoch seconds/stamps; `datetime` = calendar objects with math (`timedelta`), formatting (`strftime`) and parsing (`strptime`).

Q4. Threading skeleton?

A: `Thread(target=func, args=[...]).start()` each; `join()` all to wait. Pass data via args, collect via shared structures carefully.

Q5. Threading pitfalls?

A: Race conditions on shared mutables; GIL limits CPU parallelism (great for I/O, poor for CPU burn); always join or the program exits early.

Q6. Launch other programs?

A: `subprocess.Popen(['open', path])` / call a .py via `sys.executable`; the alarm chains `countdown` + `playsound`.

Teach it to others

The one analogy that sticks

`time.time()` is a stopwatch button; `sleep()` is a nap; threads are extra cooks in one kitchen (fast together, but they bump elbows on shared counters = race conditions); `subprocess` is ordering from another restaurant entirely.

What your students will get wrong

- x `sleep(60)` is an accurate timer (drifts - compare timestamps).
- x Threads speed up math (GIL - use multiprocessing for CPU).
- x A started thread finishes before `exit` (join it, or it dies mid-work).

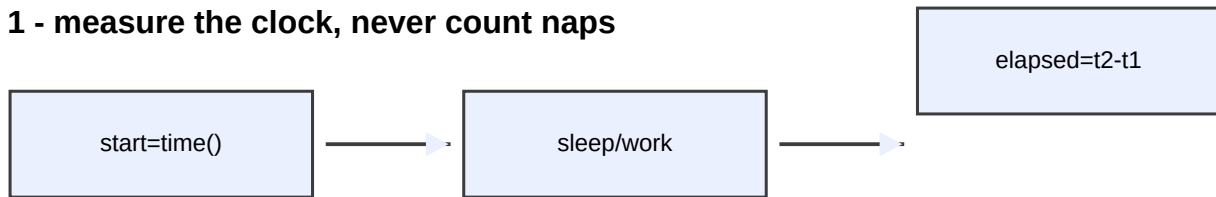
5-minute whiteboard plan

1. Time a 0.5s `sleep` vs measured elapsed.
2. Lap the stopwatch with Enter presses.
3. Race two threads appending, then discuss locks.
4. Exit ticket: parse '31/10/2015' with `strptime` format.

Visualize it - draw these from memory

Redraw each figure on paper with labels before you touch code. If you cannot draw it, you cannot teach it.

Visual 1 - measure the clock, never count naps



sleep() paces; time() measures. Countdowns compare timestamps in a loop.

Visual 2 - threads: three cooks, one kitchen



start() all, join() all. Shared counters need locks - cooks bump elbows (races).

More examples - same idea, new clothes

Three bonus drills per module. Cover the output, predict, then run.

X1. Pomodoro tick

```
import time
for left in range(3, 0, -1):
    print(left, '...')
    time.sleep(0.2)
print('Break!')
# EXPECTED: 3... 2... 1... Break!
```

X2. Days until date

```
from datetime import datetime
d = datetime.strptime('25/12/2026', '%d/%m/%Y')
print((d - datetime.now()).days, 'days left')
# EXPECTED: e.g. 78 days left
```

X3. Parallel fetchers

```
import threading, time
def job(n):
    time.sleep(0.2)
    print('done', n)
ts = [threading.Thread(target=job, args=[i]) for i in range(3)]
[t.start() for t in ts]
[t.join() for t in ts]
# EXPECTED: done 0/1/2 in ~0.2s total
```

Program map - all 37 at a glance

```
01 01_time_time_function.py
02 02_time_time_function_2.py
03 03_time_time_function_the.py
04 04_time_sleep_function.py
05 05_time_sleep_function_2.py
06 06_rounding_numbers.py
07 07_step_1_set.py
08 08_step_2_track.py
09 09_datetime_module.py
10 10_datetime_module_2.py
11 11_datetime_module_halloween201.py
12 12_timedelta_data_type.py
13 13_timedelta_data_type_dt.py
14 14_timedelta_data_type_oct21st.py
15 15_pausing_until_specific.py
16 16_converting_datetime_objects.py
17 17_converting_strings_datetime.py
18 18_multithreading.py
19 19_multithreading_2.py
20 20_multithreading_start.py
21 21_passing_arguments_thread.py
22 22_passing_arguments_thread_threading.py
23 23_passing_arguments_thread_threadobj.py
24 24_step_1_modify.py
25 25_step_2_create.py
26 26_step_3_wait.py
27 27_launching_other_programs.py
28 28_launching_other_programs_subprocess.py
29 29_launching_other_programs_calcpoc.py
30 30_passing_command_line.py
31 31_running_other_python.py
32 32_opening_files_default.py
```

33 33_opening_files_default_subprocess.py
34 34_step_1_count.py
35 35_step_2_play.py
36 36_prettified_stopwatch.py
37 37_prettified_stopwatch_lap.py

Topic: The time.time() Function

01/37 01_time_time_function.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import time
```

```
time.time()
```

```
# OUT: 1425063955.068649
```

02/37 02_time_time_function_2.py [script]

Script: save and run; predict the output first, then compare.

```
import time
def calcProd():
    # Calculate the product of the first 100,000 numbers.
    product = 1
    for i in range(1, 100000):
        product = product * i
    return product

startTime = time.time()
prod = calcProd()
endTime = time.time()
print('The result is %s digits long.' % (len(str(prod))))
print('Took %s seconds to calculate.' % (endTime - startTime))
```

03/37 03_time_time_function_the.py [script]

Script: save and run; predict the output first, then compare.

The result is 456569 digits long.

Took 2.844162940979004 seconds to calculate.

Topic: The time.sleep() Function

04/37 04_time_sleep_function.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import time
for i in range(3):
# OUT:         print('Tick')
# OUT:         time.sleep(1)
# OUT:         print('Tock')
# OUT:         time.sleep(1)
# OUT:  Tick
# OUT:  Tock
# OUT:  Tick
# OUT:  Tock
# OUT:  Tick
# OUT:  Tock
time.sleep(5)
```

05/37 05_time_sleep_function_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
for i in range(30):
```

```
# OUT:      time.sleep(1)
```

Topic: Rounding Numbers

06/37 06_rounding_numbers.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import time
now = time.time()
now
# OUT: 1425064108.017826
round(now, 2)
# OUT: 1425064108.02
round(now, 4)
# OUT: 1425064108.0178
round(now)
# OUT: 1425064108
```

Topic: Step 1: Set Up the Program to Track Times

07/37 07_step_1_set.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# stopwatch.py - A simple stopwatch program.

import time

# Display the program's instructions.
print('Press ENTER to begin. Afterwards, press ENTER to "click" the stopwatch.
Press Ctrl-C to quit.')
input()                # press Enter to begin
print('Started.')
startTime = time.time()   # get the first lap's start time
lastTime = startTime
lapNum = 1

# TODO: Start tracking the lap times.
```

Topic: Step 2: Track and Print Lap Times

08/37 08_step_2_track.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# stopwatch.py - A simple stopwatch program.

import time

--snip--

# Start tracking the lap times.
try:
    while True:
        input()
        lapTime = round(time.time() - lastTime, 2)
        totalTime = round(time.time() - startTime, 2)
        print('Lap #s: %s (%s)' % (lapNum, totalTime, lapTime), end='')
        lapNum += 1
        lastTime = time.time() # reset the last lap time
except KeyboardInterrupt:
    # Handle the Ctrl-C exception to keep its error message from displaying.
    print('\nDone.')
```

Topic: The datetime Module

09/37 09_datetime_module.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import datetime
datetime.datetime.now()
# OUT:  datetime.datetime(2015, 2, 27, 11, 10, 49, 55, 53)
dt = datetime.datetime(2015, 10, 21, 16, 29, 0)
dt.year, dt.month, dt.day
# OUT:  (2015, 10, 21)
dt.hour, dt.minute, dt.second
# OUT:  (16, 29, 0)
```

10/37 10_datetime_module_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
datetime.datetime.fromtimestamp(10000000)  
# OUT: datetime.datetime(1970, 1, 12, 5, 46, 40)  
datetime.datetime.fromtimestamp(time.time())  
# OUT: datetime.datetime(2015, 2, 27, 11, 13, 0, 604980)
```

11/37 11_datetime_module_halloween201.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
halloween2015 = datetime.datetime(2015, 10, 31, 0, 0, 0)  
newyears2016 = datetime.datetime(2016, 1, 1, 0, 0, 0)  
oct31_2015 = datetime.datetime(2015, 10, 31, 0, 0, 0)  
halloween2015 == oct31_2015  
# OUT:      True  
halloween2015 > newyears2016  
# OUT:      False  
newyears2016 > halloween2015  
# OUT:      True  
newyears2016 != oct31_2015  
# OUT:      True
```

Topic: The timedelta Data Type

12/37 12_timedelta_data_type.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
delta = datetime.timedelta(days=11, hours=10, minutes=9, seconds=8)  
delta.days, delta.seconds, delta.microseconds  
# OUT:      (11, 36548, 0)  
delta.total_seconds()  
# OUT:      986948.0  
str(delta)  
# OUT:      '11 days, 10:09:08'
```

13/37 13_timedelta_data_type_dt.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
dt = datetime.datetime.now()  
dt  
# OUT: datetime.datetime(2015, 2, 27, 18, 38, 50, 636181)  
thousandDays = datetime.timedelta(days=1000)  
dt + thousandDays  
# OUT: datetime.datetime(2017, 11, 23, 18, 38, 50, 636181)
```

14/37 14_timedelta_data_type_oct21st.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
oct21st = datetime.datetime(2015, 10, 21, 16, 29, 0)  
aboutThirtyYears = datetime.timedelta(days=365 * 30)  
oct21st  
# OUT:      datetime.datetime(2015, 10, 21, 16, 29)  
oct21st - aboutThirtyYears  
# OUT:      datetime.datetime(1985, 10, 28, 16, 29)  
oct21st - (2 * aboutThirtyYears)  
# OUT:      datetime.datetime(1955, 11, 5, 16, 29)
```

Topic: Pausing Until a Specific Date

15/37 15_pausing_until_specific.py [script]

Script: save and run; predict the output first, then compare.

```
import datetime
import time
halloween2016 = datetime.datetime(2016, 10, 31, 0, 0, 0)
while datetime.datetime.now() < halloween2016:
    time.sleep(1)
```

Topic: Converting datetime Objects into Strings

16/37 16_converting_datetime_objects.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
oct21st = datetime.datetime(2015, 10, 21, 16, 29, 0)  
oct21st.strftime('%Y/%m/%d %H:%M:%S')  
# OUT: '2015/10/21 16:29:00'  
oct21st.strftime('%I:%M %p')  
# OUT: '04:29 PM'  
oct21st.strftime("%B of '%y")  
# OUT: "October of '15"
```

Topic: Converting Strings into datetime Objects

17/37 17_converting_strings_datetime.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
# Lines starting with >>> are input; other lines are expected output (commented).

datetime.datetime.strptime('October 21, 2015', '%B %d, %Y')
# OUT:    datetime.datetime(2015, 10, 21, 0, 0)
datetime.datetime.strptime('2015/10/21 16:29:00', '%Y/%m/%d %H:%M:%S')
# OUT:    datetime.datetime(2015, 10, 21, 16, 29)
datetime.datetime.strptime("October of '15", "%B of '%y")
# OUT:    datetime.datetime(2015, 10, 1, 0, 0)
datetime.datetime.strptime("November of '63", "%B of '%y")
# OUT:    datetime.datetime(2063, 11, 1, 0, 0)
```

Topic: Multithreading

18/37 18_multithreading.py [script]

Script: save and run; predict the output first, then compare.

```
import time, datetime

startTime = datetime.datetime(2029, 10, 31, 0, 0, 0)
while datetime.datetime.now() < startTime:
    time.sleep(1)

print('Program now starting on Halloween 2029')
--snip--
```

19/37 19_multithreading_2.py [script]

Script: save and run; predict the output first, then compare.

```
import threading, time
print('Start of program.')

def takeANap():
    time.sleep(5)
    print('Wake up!')

threadObj = threading.Thread(target=takeANap)
threadObj.start()

print('End of program.')
```

20/37 20_multithreading_start.py [script]

Script: save and run; predict the output first, then compare.

Start of program.

End of program.

Wake up!

Topic: Passing Arguments to the Thread's Target Function

21/37 21_passing_arguments_thread.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
print('Cats', 'Dogs', 'Frogs', sep=' & ')
```

```
# OUT: Cats & Dogs & Frogs
```

22/37 22_passing_arguments_thread_threading.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
import threading  
threadObj = threading.Thread(target=print, args=['Cats', 'Dogs', 'Frogs'],  
# OUT: kwargs={'sep': ' & '})  
threadObj.start()  
# OUT: Cats & Dogs & Frogs
```

23/37 23_passing_arguments_thread_threadobj.py [script]

Script: save and run; predict the output first, then compare.

```
threadObj = threading.Thread(target=print('Cats', 'Dogs', 'Frogs', sep=' & '))
```

Topic: Step 1: Modify the Program to Use a Function

24/37 24_step_1_modify.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# multidownloadXkcd.py - Downloads XKCD comics using multiple threads.

import requests, os, bs4, threading
os.makedirs('xkcd', exist_ok=True) # store comics in ./xkcd

def downloadXkcd(startComic, endComic):
    for urlNumber in range(startComic, endComic + 1):
        # Download the page.
        print('Downloading page http://xkcd.com/%s...' % (urlNumber))
        res = requests.get('http://xkcd.com/%s' % (urlNumber))
        res.raise_for_status()

        soup = bs4.BeautifulSoup(res.text)

        # Find the URL of the comic image.
        comicElem = soup.select('#comic img')
        if comicElem == []:
            print('Could not find comic image.')
        else:
            comicUrl = comicElem[0].get('src')
            # Download the image.
            print('Downloading image %s...' % (comicUrl))
            res = requests.get(comicUrl)
            res.raise_for_status()

            # Save the image to ./xkcd.
            imageFile = open(os.path.join('xkcd', os.path.basename(comicUrl)), 'wb')
            for chunk in res.iter_content(1000000):
                imageFile.write(chunk)
            imageFile.close()

# TODO: Create and start the Thread objects.
# TODO: Wait for all threads to end.
```

Topic: Step 2: Create and Start Threads

25/37 25_step_2_create.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# multidownloadXkcd.py - Downloads XKCD comics using multiple threads.

--snip--

# Create and start the Thread objects.
downloadThreads = []          # a list of all the Thread objects
for i in range(0, 1400, 100):  # loops 14 times, creates 14 threads
    downloadThread = threading.Thread(target=downloadXkcd, args=(i, i + 99))
    downloadThreads.append(downloadThread)
    downloadThread.start()
```

Topic: Step 3: Wait for All Threads to End

26/37 26_step_3_wait.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# multidownloadXkcd.py - Downloads XKCD comics using multiple threads.

--snip--

# Wait for all threads to end.
for downloadThread in downloadThreads:
    downloadThread.join()
print('Done.')
```

Topic: Launching Other Programs from Python

27/37 27_launching_other_programs.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import subprocess
```

```
subprocess.Popen('C:\\Windows\\System32\\calc.exe')
```

```
# OUT: <subprocess.Popen object at 0x000000003055A58>
```

28/37 28_launching_other_programs_subprocess.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
import subprocess
```

```
subprocess.Popen('/usr/bin/gnome-calculator')
```

```
# OUT: <subprocess.Popen object at 0x7f2bcf93b20>
```

29/37 29_launching_other_programs_calcpoc.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.  
# Lines starting with >>> are input; other lines are expected output (commented).  
  
calcProc = subprocess.Popen('c:\\Windows\\System32\\calc.exe')  
calcProc.poll() == None  
# OUT:    True  
calcProc.wait()  
# OUT:    0  
calcProc.poll()  
# OUT:    0
```

Topic: Passing Command Line Arguments to Popen()

30/37 30_passing_command_line.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
subprocess.Popen(['C:\\windows\\notepad.exe', 'C:\\hello.txt'])
```

```
# OUT: <subprocess.Popen object at 0x00000000032DCEB8>
```

Topic: Running Other Python Scripts

31/37 31_running_other_python.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
subprocess.Popen(['C:\\python34\\python.exe', 'hello.py'])
```

```
# OUT: <subprocess.Popen object at 0x00000000331CF28>
```

Topic: Opening Files with Default Applications

32/37 32_opening_files_default.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
fileObj = open('hello.txt', 'w')
fileObj.write('Hello world!')
# OUT: 12
fileObj.close()
import subprocess
subprocess.Popen(['start', 'hello.txt'], shell=True)
```

33/37 33_opening_files_default_subprocess.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

```
# Interactive session from the book. Run line-by-line in IDLE.
```

```
# Lines starting with >>> are input; other lines are expected output (commented).
```

```
subprocess.Popen(['open', '/Applications/Calculator.app/'])
```

```
# OUT: <subprocess.Popen object at 0x10202ff98>
```

Topic: Step 1: Count Down

34/37 34_step_1_count.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# countdown.py - A simple countdown script.

import time, subprocess

timeLeft = 60
while timeLeft > 0:
    print(timeLeft, end='')
    time.sleep(1)
    timeLeft = timeLeft - 1

# TODO: At the end of the countdown, play a sound file.
```

Topic: Step 2: Play the Sound File

35/37 35_step_2_play.py [script]

Script: save and run; predict the output first, then compare.

```
#!/ python3
# countdown.py - A simple countdown script.

import time, subprocess

--snip--

# At the end of the countdown, play a sound file.
subprocess.Popen(['start', 'alarm.wav'], shell=True)
```

Topic: Prettified Stopwatch

36/37 36_prettified_stopwatch.py [script]

Script: save and run; predict the output first, then compare.

```
Lap #1: 3.56 (3.56)
Lap #2: 8.63 (5.07)
Lap #3: 17.68 (9.05)
Lap #4: 19.11 (1.43)
```

37/37 37_prettified_stopwatch_lap.py [script]

Script: save and run; predict the output first, then compare.

```
Lap # 1: 3.56 ( 3.56)
Lap # 2: 8.63 ( 5.07)
Lap # 3: 17.68 ( 9.05)
Lap # 4: 19.11 ( 1.43)
```